Spring Day03

```
class ProductDao{

  private static final String SQL_QUERY_FOR_COUNT="select count(1) from Product";
  private JdbcTemplate template;

   ProductDao(JdbcTemplate template){
        this.template=template;
    }
    //now let's see how we can perform the operation

    public int getNoOfProduct(){
        template.queryForObject(SQL_QUERY_FOR_COUNT,Integer.class);
    }
}
```

application-context.xml

```
<beans>
   <bean id="datasource" class="org.springframework.jdbc.datasource.DataSource">
       // setting the driver name, url ,username and password
    </bean>

 <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate>
     <constructor-args ref="datasource"/>
</bean>

 <bean id="productdao" class="com.day35.Product">
    <constructor - args ref="template"/>
</bean>
</beans>        so this will established all the connection
```

let's see now how it's internally works

```
class JdbcTemplate{
    private DataSource datasource;

    public JdbcTemplate(DataSource datasource){
        this.datasource=datasource;
    }

  public Object queryForObject(String sql, Class<?> classType){
        Connection con=null;
        Statement st=null;
        ResultSet rs=null;
        Object result=null;

        try{
            con=datasource.getConnection();
            st=con.createStatement(sql);
            rs=st.executeQuer(sq);

            while(rs.next()){
                if(classType== Integer.class){
                    result=rs.getInt(1);
                }
                else if(classType==String.class){
                    result=rs.getString(1);
                }
                else if(classType==Float.class){
                    result=rs.getFloat(1);
                }
            }
            return result;
        }
    }
```

this is the internal for how queryForObject() is working

let's do a practical for it

With the xml approach

so 1st of all we should have to add the dependencies of JDBC driver and jdbc connector in our pom.xml along with spring-core and spring-context

Directroy structure is


```
|- ProductDao class
|- application-context.xml
|- Main
|
|
|- pom.xml
```

so let' see in below pom.xml dependencies

we can take these dependencies from mvn repository where we can search ojdbc8 and we can get the jar dependancy

```
<dependency>
  <groupId>com.oracle.database.jdbc</groupId>
  <artifactId>ojdbc8</artifactId>
  <version>23.8.0.25.04</version>
  <scope>compile</scope>
</dependency>
```

for the spring jdbc dependency you can search spring-jdbc and take the same dependancy version as spring-core and spring-context

```
<!-- Source: https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-jdbc</artifactId>
  <version>6.2.7</version>
  <scope>compile</scope>
</dependency>
```

after adding all these dependencies along with the spring core and spring context now we will update the project with the help of Maven
|- updateProject

Now let' build the project based on xml approach

Product.java

```
package com.day35;
import org.springframework.jdbc.core.JdbcTemplate;

public class Product {
  private final static String
     SQL_QUERY_FOR_COUNT="select count(1) from Product";

  JdbcTemplate template;

  public Product(JdbcTemplate template) {
    super();
    this.template = template;
  }
  @Override
  public String toString() {
    return "Product [template=" + template + "]";
  }

   public int getNoOfProduct() {
     return template.queryForObject(SQL_QUERY_FOR_COUNT, Integer.class);
   }
}
```

application.context.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns:context="http://www.springframework.org/schema/context"

 xsi:schemaLocation="
      http://www.springframework.org/schema/beans
      http://www.springframework.org/schema/beans/spring-beans.xsd
      http://www.springframework.org/schema/context
      https://www.springframework.org/schema/context/spring-context.xsd">

 <bean id="dataSource"
  class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName"
    value="oracle.jdbc.driver.OracleDriver">
  </property>
  <property name="url"
    value="jdbc:oracle:thin:@localhost:1521:xe">
  </property>
  <property name="username" value="system">
  </property>
  <property name="password" value="manager">
  </property>
 </bean>

 <bean id="jdbcTemplate"
  class="org.springframework.jdbc.core.JdbcTemplate">
   <constructor-arg ref="dataSource"/>
 </bean>

 <bean id="product"
        class="com.day35.Product">
         <constructor-arg ref="jdbcTemplate"/>
 </bean>

</beans>
```

Main.java

```
package com.day35;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Main {
public static void main(String[] args) {
  ApplicationContext context=new
      ClassPathXmlApplicationContext("com/day34/application-context.xml");
      Product bean = context.getBean("product",Product.class);
     System.out.println("No of record present inside the column ");
     System.out.println(bean.getNoOfProduct());
}
}
```

Now let's build the project based on javaConfiguration approach

Product.java

```
package com.day35;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

@Repository("product")
public class ProductDao {

  private final static String SQL_QUERY_FOR_COUNT="select count(1) from Product";
  private JdbcTemplate template;

  @Autowired
  public ProductDao(JdbcTemplate template) {
    super();
    this.template = template;
  }

  public int getNumberOfProduct() {
    return template.queryForObject(SQL_QUERY_FOR_COUNT,Integer.class);
  }


}
```

JavaConfig.class

```
package com.day35;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;
import org.springframework.core.env.Environment;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

@Configuration
@ComponentScan(basePackages = "com.day35")
@PropertySource("classpath:com/day35/application.properties")
public class JavaConfig {

 @Autowired
 Environment environment;

 @Bean
 public DataSource dataSource() {
    DriverManagerDataSource source=
        new DriverManagerDataSource();

    source.setDriverClassName(environment.getProperty("db.driverName"));
    source.setUrl(environment.getProperty("db.url"));
    source.setUsername(environment.getProperty("db.username"));
    source.setPassword(environment.getProperty("db.password"));
    return source;
 }

 @Bean
 public JdbcTemplate template() {
    JdbcTemplate template=new JdbcTemplate(dataSource());
    return template;

 }
}
```

application.properties

```
db.driverName=oracle.jdbc.driver.OracleDriver
db.url=jdbc:oracle:thin:@localhost:1521:xe
db.username=system
db.password=manager
```

Main.java

```
package com.day35;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
  public static void main(String[] args) {
    AnnotationConfigApplicationContext context =
        new AnnotationConfigApplicationContext(JavaConfig.class);

    ProductDao bean = context
        .getBean("product", ProductDao.class);
    System.out.println("Number of Product present in the table");
    System.out.println(bean.getNumberOfProduct());
  }
}
```

SAME CODE FOR GETTING THE PRODUCT NAME BASED ON Product ID

In the same ProductDao.java

add the below variable as a Sql query

```
private final static String SQL_QUERY_FOR_NAME
    ="select Productname from Product where ProductId=?";
```

as well as

create one more method for getting the product name based on ID

```
public String getNameOfProduct(int productid) {
    return template.queryForObject
        (SQL_QUERY_FOR_NAME, String.class,
                    new Object[] { productid});
}
```

and inside the Main class

call the method inside the Sysout

```
System.out.println("Name :"+bean.getNameOfProduct(101));
```